

CLOUD COMPUTING

LAB NO # 11



SUBMITTED TO:

Sir Shohaib

SUBMITTED BY:

Name: Amina Noor

Reg No. 2023-BSE-007

Semester: V-A

CLOUD COMPUTING

LAB NO # 11

TASK 0 — Lab Setup (Codespace & GH CLI)

Step 1: Open a Codespace-enabled environment

You have **two valid ways**:

✓ Option A (Recommended): GitHub Codespaces Web

1. Open **GitHub**
2. Click **New repository**
 - Name: CC_AminaNoor_<rollNumber>/Lab11
👉 *Use EXACT format given by your teacher*
 - Public
 - Create repo
3. Click **Code** → **Codespaces** → **Create codespace**

Step 2: Verify GH CLI is installed (inside Codespace)

Run: `gh --version`

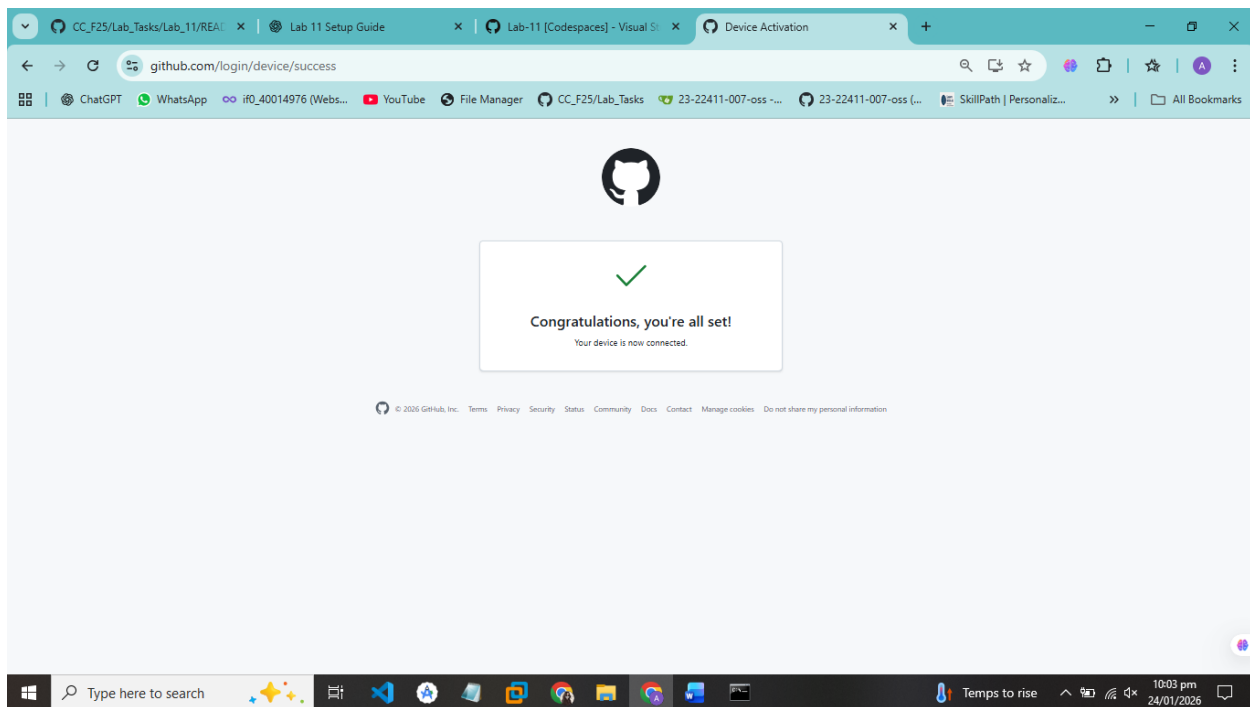
```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ gh --version
gh version 2.83.1 (2025-11-13)
https://github.com/cli/cli/releases/tag/v2.83.1
```

Step 3: Authenticate GH CLI (INSIDE Codespace)

`gh auth login`

Choose:

- GitHub.com
- HTTPS
- Login with browser
- Copy code → open link → authorize



Verify: gh auth status

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ gh auth status
github.com
✓ Logged in to github.com account 23-22411-007-oss (GITHUB_TOKEN)
- Active account: true
- Git operations protocol: https
- Token: ghu_*****
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

Step 4: Create Codespace using GH CLI

```
gh codespace create --repo <your-github-username>/Lab11
```

Then list:

```
gh codespace list
```

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ gh codespace list
```

NAME	DISPLAY NAME	REPOSITORY	BRANCH	STATE	CREATED AT
obscure-spork-97g9...	obscure spork	23-22411-007-oss...	main	Available	about 37 minutes...

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

Step 5: SSH into the Codespace

```
gh codespace ssh -c <your_codespace_name>
```

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ gh codespace ssh -c obscure-spork-97g9w96p7jv5cpr
x4
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

TASK 1 — Provider & Basic Variable (STEP-BY-STEP)

◆ Step 1: Create main.tf

Run:

touch main.tf

ls -l

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ touch main.tf
ls -l
total 4
-rw-rw-rw- 1 codespace root      127 Jan 24 16:44 README.md
-rw-rw-rw- 1 codespace codespace  0 Jan 24 17:26 main.tf
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

◆ Step 2: Add AWS provider to main.tf

Open editor:

code main.tf

Paste **exactly this**:

```
provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}
```

Save the file (Ctrl+S).

```

GNU nano 7.2                                main.tf
provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}

```

◆ Step 3: Initialize Terraform

Run:

terraform init

```

@23-22411-007-oss → /workspaces/Lab-11 (main) $ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v6.28.0...
- Installed hashicorp/aws v6.28.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
@23-22411-007-oss → /workspaces/Lab-11 (main) $

```

◆ Step 4: Add variable and output

Open file again:

code main.tf

Append **below the provider**:

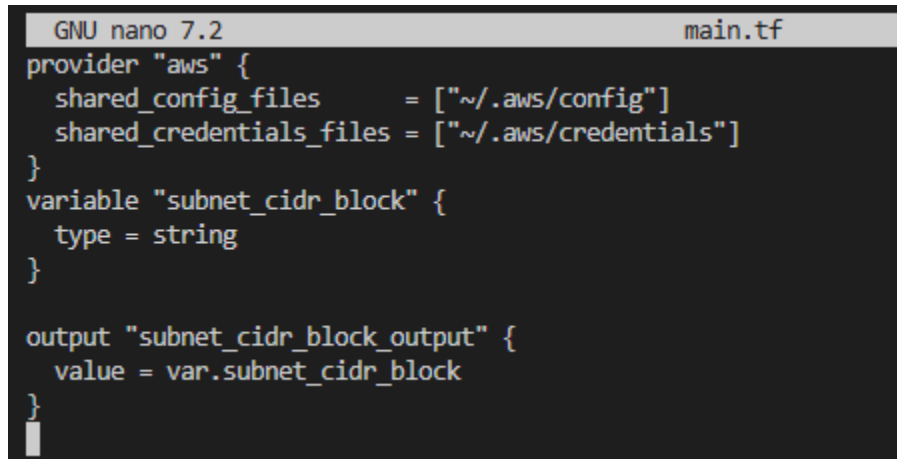
```

variable "subnet_cidr_block" {
  type = string
}

```

```
output "subnet_cidr_block_output" {  
  value = var.subnet_cidr_block  
}
```

Save.



```
GNU nano 7.2 main.tf  
provider "aws" {  
  shared_config_files    = ["~/.aws/config"]  
  shared_credentials_files = ["~/.aws/credentials"]  
}  
variable "subnet_cidr_block" {  
  type = string  
}  
  
output "subnet_cidr_block_output" {  
  value = var.subnet_cidr_block  
}
```

◆ Step 5: Apply WITHOUT default (prompt expected)

Run:

```
terraform apply -auto-approve
```

Terraform will ask:

```
var.subnet_cidr_block
```

Enter a value:

Type:

```
10.0.10.0/24
```

Press **Enter**.

```

@23-22411-007-oss →/workspaces/Lab-11 (main) $ terraform apply -auto-approve
var.subnet_cidr_block
  Enter a value: 10.0.10.0/24

Changes to Outputs:
  + subnet_cidr_block_output = "10.0.10.0/24"

You can apply this plan to save these new output values to the Terraform state, without changing
any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

subnet_cidr_block_output = "10.0.10.0/24"
@23-22411-007-oss →/workspaces/Lab-11 (main) $

```

◆ Step 6: Add DEFAULT value

Edit main.tf:

code main.tf

Replace variable block with:

```

variable "subnet_cidr_block" {
  type    = string
  default = "10.0.0.0/24"
}

```

Save.

```

GNU nano 7.2 main.tf *
provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}
variable "subnet_cidr_block" {
  type    = string
  default = "10.0.0.0/24"
}

output "subnet_cidr_block_output" {
  value = var.subnet_cidr_block
}

```

Run:

terraform apply -auto-approve

```

@23-22411-007-oss →/workspaces/Lab-11 (main) $ terraform apply -auto-approve

Changes to Outputs:
  ~ subnet_cidr_block_output = "10.0.10.0/24" -> "10.0.0.0/24"

You can apply this plan to save these new output values to the Terraform state, without changing
any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
subnet_cidr_block_output = "10.0.0.0/24"
@23-22411-007-oss →/workspaces/Lab-11 (main) $

```

◆ Step 7: Environment variable precedence

In terminal:

export TF_VAR_subnet_cidr_block=10.0.20.0/24

terraform apply -auto-approve

```

@23-22411-007-oss →/workspaces/Lab-11 (main) $ export TF_VAR_subnet_cidr_block=10.0.20.0/24
terraform apply -auto-approve

Changes to Outputs:
  ~ subnet_cidr_block_output = "10.0.0.0/24" -> "10.0.20.0/24"

You can apply this plan to save these new output values to the Terraform state, without changing
any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
subnet_cidr_block_output = "10.0.20.0/24"
@23-22411-007-oss →/workspaces/Lab-11 (main) $

```

◆ Step 8: terraform.tfvars (higher precedence)

Create file:

touch terraform.tfvars

Edit:

code terraform.tfvars

Add:

subnet_cidr_block = "10.0.30.0/24"

Save.


```
GNU nano 7.2 terraform.tfvars *
subnet_cidr_block = "10.0.30.0/24"

```

Run:

cat terraform.tfvars

terraform apply -auto-approve

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ cat terraform.tfvars
terraform apply -auto-approve
subnet_cidr_block = "10.0.30.0/24"

Changes to Outputs:
  ~ subnet_cidr_block_output = "10.0.20.0/24" -> "10.0.30.0/24"

You can apply this plan to save these new output values to the Terraform state, without changing
any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

subnet_cidr_block_output = "10.0.30.0/24"
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

◆ Step 9: -var (highest precedence)

Run:

terraform apply -auto-approve -var "subnet_cidr_block=10.0.40.0/24"

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ terraform apply -auto-approve -var "subnet_cidr_b
lock=10.0.40.0/24"

Changes to Outputs:
  ~ subnet_cidr_block_output = "10.0.30.0/24" -> "10.0.40.0/24"

You can apply this plan to save these new output values to the Terraform state, without changing
any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

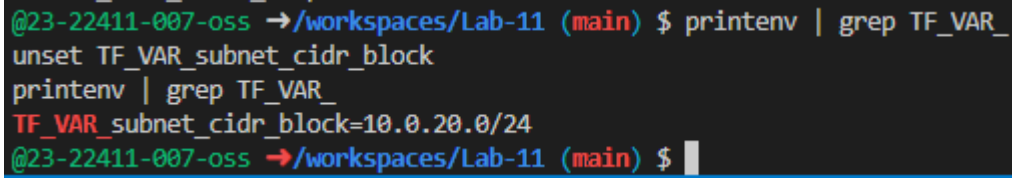
Outputs:

subnet_cidr_block_output = "10.0.40.0/24"
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

◆ Step 10: Show and unset environment variable

Run:

```
printenv | grep TF_VAR_  
unset TF_VAR_subnet_cidr_block  
printenv | grep TF_VAR_
```



```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ printenv | grep TF_VAR_  
unset TF_VAR_subnet_cidr_block  
printenv | grep TF_VAR_  
TF_VAR_subnet_cidr_block=10.0.20.0/24  
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

TASK 2 — Variable Validation & Sensitive / Ephemeral Variables

◆ STEP 1: Replace subnet_cidr_block variable (with validation)

Open file:

```
nano main.tf
```

🗑 DELETE the old subnet_cidr_block variable block

Then paste **EXACTLY THIS**:

```
variable "subnet_cidr_block" {  
    type      = string  
    default   = ""  
    description = "CIDR block to assign to the application subnet"  
    sensitive  = false  
    nullable   = false  
    ephemeral  = false  
  
    validation {  
        condition = can(regex("^[0-9]{1,3}\\.{3}[0-9]{1,3}/[0-9]+$", var.subnet_cidr_block))  
        error_message = "The subnet_cidr_block must be a valid CIDR notation string, such as  
10.0.0.0/24."  
    }  
}
```

Keep provider + outputs as-is.

Save:

- Ctrl + O → Enter
- Ctrl + X

```
GNU nano 7.2                                main.tf *
provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}
variable "subnet_cidr_block" {
  type          = string
  default       = ""
  description   = "CIDR block to assign to the application subnet"
  sensitive     = false
  nullable     = false
  ephemeral    = false

  validation {
    condition     = can(regex("^[0-9]{1,3}\\.{3}[0-9]{1,3}/[0-9]+$", var.subnet_cidr_block))
    error_message = "The subnet_cidr_block must be a valid CIDR notation string, such as 10.0.0"
  }
}

output "subnet_cidr_block_output" {
  value = var.subnet_cidr_block
}
```

◆ STEP 2: Test validation FAILURE

Run:

```
terraform apply -auto-approve -var "subnet_cidr_block=10.0.0"
```

✗ This is **INVALID CIDR**, Terraform should stop with:

Error: Invalid value for variable

The subnet_cidr_block must be a valid CIDR notation string

```
@23-22411-007-oss → /workspaces/Lab-11 (main) $ terraform apply -auto-approve -var "subnet_cidr_block=10.0.0"

Error: Invalid value for variable

on main.tf line 5:
5: variable "subnet_cidr_block" {
    | var.subnet_cidr_block is "10.0.0"

The subnet_cidr_block must be a valid CIDR notation string, such as 10.0.0.0/24.

This was checked by the validation rule at main.tf:13,3-13.
```

◆ STEP 3: Add sensitive variable `api_session_token`

Open file:

```
nano main.tf
```

⬇ ADD BELOW subnet variable:

```
variable "api_session_token" {  
    type      = string  
    default   = ""  
  
    description = "Short-lived API session token used during apply operations"  
  
    sensitive  = true  
    nullable   = false  
    ephemeral  = false  
  
    validation {  
        condition = can(regex("^[A-Za-z0-9-_{20,}$", var.api_session_token))  
        error_message = "The API session token must be at least 20 characters and contain only  
letters, numbers, hyphens, or underscores."  
    }  
}
```

⬇ ADD OUTPUT (sensitive):

```
output "api_session_token_output" {  
    value     = var.api_session_token  
    sensitive = true  
}
```

Save and exit.

```

variable "api_session_token" {
  type      = string
  default   = ""
  description = "Short-lived API session token used during apply operations"
  sensitive = true
  nullable  = false
  ephemeral = false

  validation {
    condition     = can(regex("^[A-Za-z0-9-]{20,}$", var.api_session_token))
    error_message = "The API session token must be at least 20 characters and contain only letters, numbers, hyph"
  }
}
output "api_session_token_output" {
  value     = var.api_session_token
  sensitive = true
}

```

◆ STEP 4: Apply with sensitive variable

Run:

```
terraform apply -auto-approve \
```

```
-var "subnet_cidr_block=10.0.0.0/24" \
```

```
-var "api_session_token=my_API_session_Token_12345"
```

```

@23-22411-007-oss →/workspaces/Lab-11 (main) $ terraform apply -auto-approve \
  -var "subnet_cidr_block=10.0.0.0/24" \
  -var "api_session_token=my_API_session_Token_12345"

```

Changes to Outputs:

```

+ api_session_token_output = (sensitive value)
~ subnet_cidr_block_output = "10.0.40.0/24" -> "10.0.0.0/24"

```

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

```

api_session_token_output = <sensitive>
subnet_cidr_block_output = "10.0.0.0/24"

```

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

◆ STEP 5: Check terraform state file

Run:

```
grep -A5 api_session_token_output terraform.tfstate
```

You should see something like:

```

"api_session_token_output": {
  "value": "my_API_session_Token_12345",

```

```
"type": "string",  
"sensitive": true  
}
```

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ grep -A5 api_session_token_output terraform.tfstate  
  "api_session_token_output": {  
    "value": "my_API_session_Token_12345",  
    "type": "string",  
    "sensitive": true  
  },  
  "subnet_cidr_block_output": {  
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

◆ STEP 6: Make variable EPHEMERAL (expected failure / behavior)

Edit: nano main.tf

Change: ephemeral = false

to: ephemeral = true

Save.

```
variable "subnet_cidr_block" {  
  type      = string  
  default   = ""  
  description = "CIDR block to assign to the application subnet"  
  sensitive  = false  
  nullable   = false  
  ephemeral  = true  
  
  validation {  
    condition     = can(regex("^[0-9]{1,3}\\.{3}[0-9]{1,3}/[0-9]+$", var.subnet_cidr_block))  
    error_message = "The subnet_cidr_block must be a valid CIDR notation string, such as 10.0.0.0/24."  
  }  
}  
  
variable "api_session_token" {  
  type      = string  
  default   = ""  
  description = "Short-lived API session token used during apply operations"  
  sensitive  = true  
  nullable   = false  
  ephemeral  = true
```

Run:

terraform apply -auto-approve \

-var "subnet_cidr_block=10.0.0.0/24" \

-var "api_session_token=my_API_session_Token_12345"

⚠ Terraform will complain about:

- Ephemeral value used in output

- Or not being stored in state

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ terraform apply -auto-approve \
-var "subnet_cidr_block=10.0.0.0/24" \
-var "api_session_token=my_API_session_Token_12345"

Error: Ephemeral value not allowed

on main.tf line 32, in output "api_session_token_output":
32:   value      = var.api_session_token

This output value is not declared as returning an ephemeral value, so it cannot be set to a result derived from
an ephemeral value.

Error: Ephemeral value not allowed

on main.tf line 37, in output "subnet_cidr_block_output":
37:   value = var.subnet_cidr_block

This output value is not declared as returning an ephemeral value, so it cannot be set to a result derived from
an ephemeral value.

@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

variable (stored in state)

Edit again:

nano main.tf

Update variable to:

```
variable "api_session_token" {
  type      = string
  default   = "my_API_session_Token"
  sensitive = true
  ephemeral = false
  nullable  = false
}
```

```
}
}
variable "api_session_token" {
  type      = string
  default   = "my_API_session_Token"
  sensitive = true
  ephemeral = false
  nullable  = false
}
```

Save.

Run: terraform apply -auto-approve \

-var "subnet_cidr_block=10.0.0.0/24"

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ terraform apply -auto-approve \
-var "subnet_cidr_block=10.0.0.0/24" \
-var "api_session_token=my_API_session_Token_12345"

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no
changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

api_session_token_output = <sensitive>
subnet_cidr_block_output = "10.0.0.0/24"
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

TASK 3 — Project-level variables, locals, and outputs

◆ Step 1: Add new variables to main.tf

Open main.tf:

nano main.tf

Scroll to the **bottom of your variable blocks**, then add:

```
variable "environment" {}
```

```
variable "project_name" {}
```

```
variable "primary_subnet_id" {}
```

```
variable "subnet_count" {}
```

```
variable "monitoring" {}
```

Save the file (Ctrl+O → Enter, Ctrl+X).


```

GNU nano 7.2                                main.tf *
variable "api_session_token" {
  type      = string
  default   = ""
  description = "Short-lived API session token used during apply operations"
  sensitive = true
  nullable  = false

  validation {
    condition = can(regex("^[A-Za-z0-9-_{20,}$", var.api_session_token))
    error_message = "The API session token must be at least 20 characters and contain on
  }
}

# Task 3 variables
variable "environment" {}
variable "project_name" {}
variable "primary_subnet_id" {}
variable "subnet_count" {}
variable "monitoring" {}

# Outputs

```

◆ Step 2: Discover actual subnet ID for me-central-1a

Run:

```
aws ec2 describe-subnets \
```

```
--filters "Name=availability-zone,Values=me-central-1a" \
```

```
--query "Subnets[].SubnetId" \
```

```
--output text
```

Access key = AKIATFX3NIVRDWZNCPXB

Secret key = 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

Subnet-id = subnet-0ed9f14bcd5dfebb3

```

@23-22411-007-oss → /workspaces/Lab-11 (main) $ nano main.tf
@23-22411-007-oss → /workspaces/Lab-11 (main) $ aws ec2 describe-subnets \
--filters "Name=availability-zone,Values=me-central-1a" \
--query "Subnets[].SubnetId" \
--output text
subnet-0ed9f14bcd5dfebb3
@23-22411-007-oss → /workspaces/Lab-11 (main) $

```

◆ Step 3: Populate terraform.tfvars

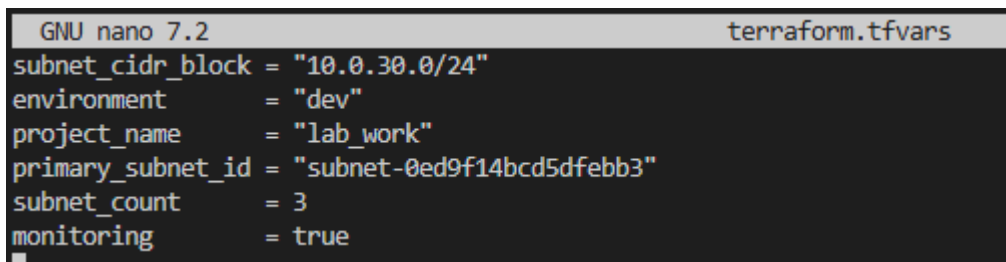
If you don't have the file yet:

```
nano terraform.tfvars
```

Paste the following (replace <subnet-id-of-me-central-1a> with the actual ID you got):

```
environment    = "dev"
project_name    = "lab_work"
primary_subnet_id = "<subnet-id-of-me-central-1a>"
subnet_count    = 3
monitoring      = true
```

Save (Ctrl+O → Enter, Ctrl+X).

A screenshot of a terminal window with a dark background. The title bar at the top shows "GNU nano 7.2" on the left and "terraform.tfvars" on the right. The terminal content displays the following text:

```
subnet_cidr_block = "10.0.30.0/24"
environment        = "dev"
project_name       = "lab_work"
primary_subnet_id  = "subnet-0ed9f14bcd5dfebb3"
subnet_count       = 3
monitoring         = true
```

◆ Step 4: Create locals.tf

```
nano locals.tf
```

Paste:

```
locals {
  resource_name    = "${var.project_name}-${var.environment}"
  primary_public_subnet = var.primary_subnet_id
  subnet_count      = var.subnet_count
  is_production     = var.environment == "prod"
  monitoring_enabled = var.monitoring || local.is_production
}
```

Save (Ctrl+O → Enter, Ctrl+X).

```
GNU nano 7.2                                locals.tf
locals {
  resource_name      = "${var.project_name}-${var.environment}"
  primary_public_subnet = var.primary_subnet_id
  subnet_count       = var.subnet_count
  is_production       = var.environment == "prod"
  monitoring_enabled  = var.monitoring || local.is_production
}
```

◆ Step 5: Add outputs to main.tf

Open main.tf again:

```
nano main.tf
```

At the **bottom of the file**, add:

```
output "resource_name" {
  value = local.resource_name
}

output "primary_public_subnet" {
  value = local.primary_public_subnet
}

output "subnet_count" {
  value = local.subnet_count
}

output "is_production" {
  value = local.is_production
}

output "monitoring_enabled" {
  value = local.monitoring_enabled
}
```

Save (Ctrl+O → Enter, Ctrl+X).

```
GNU nano 7.2 main.tf *
}

output "api_session_token_output" {
  value = var.api_session_token
  sensitive = true
}

output "resource_name" {
  value = local.resource_name
}

output "primary_public_subnet" {
  value = local.primary_public_subnet
}

output "subnet_count" {
  value = local.subnet_count
}

output "is_production" {
  value = local.is_production
}

output "monitoring_enabled" {
  value = local.monitoring_enabled
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify
```

◆ Step 6: Run Terraform Apply

terraform apply -auto-approve

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ terraform apply -auto-approve \
-var "api_session_token=my_API_session_Token_12345"

Changes to Outputs:
+ is_production          = false
+ monitoring_enabled     = true
+ primary_public_subnet  = "subnet-0ed9f14bcd5dfebb3"
+ resource_name          = "lab_work-dev"
~ subnet_cidr_block_output = "10.0.0.0/24" -> "10.0.30.0/24"
+ subnet_count           = 3

You can apply this plan to save these new output values to the Terraform state, without changing any real
infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

api_session_token_output = <sensitive>
is_production            = false
monitoring_enabled       = true
primary_public_subnet    = "subnet-0ed9f14bcd5dfebb3"
resource_name            = "lab_work-dev"
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count             = 3
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

TASK 4 — Maps and Objects

◆ Step 1: Add tags map variable in main.tf

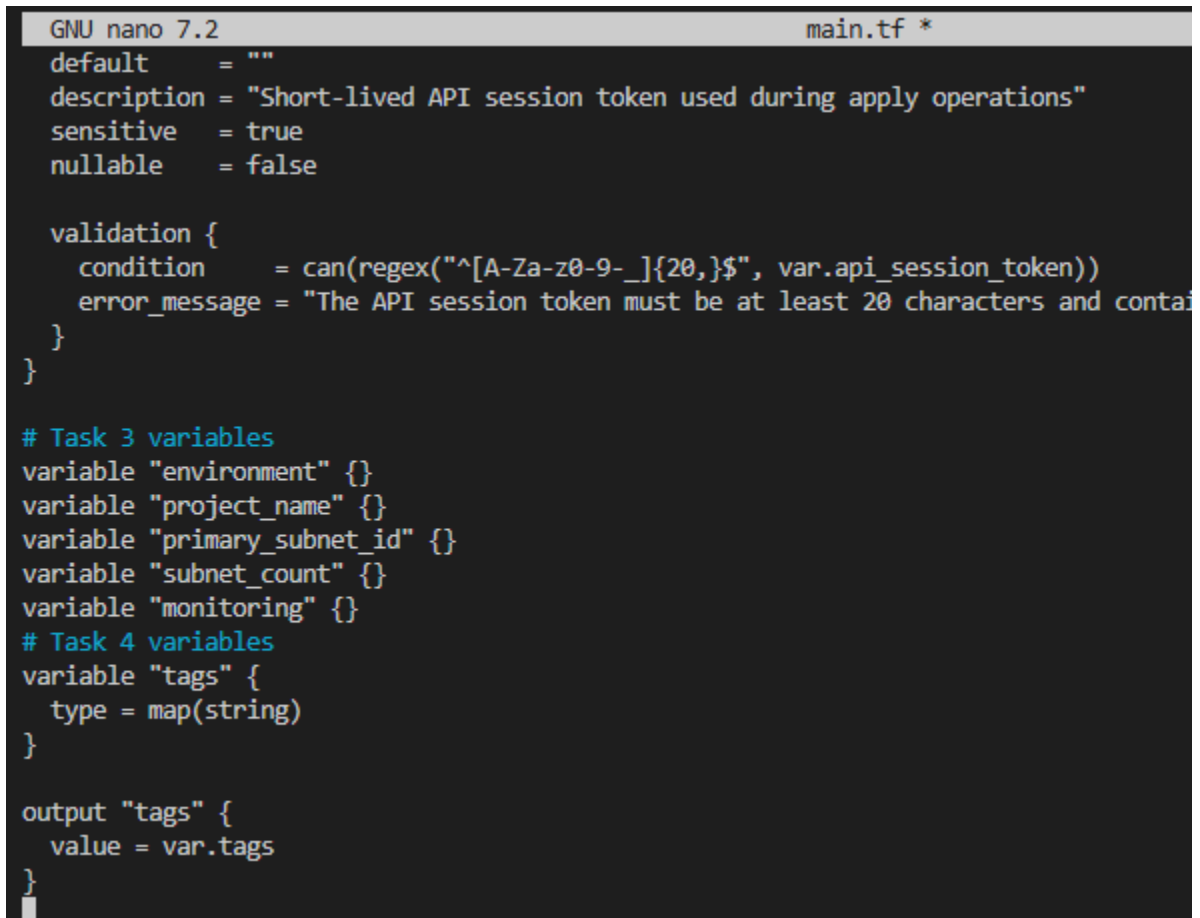
Open main.tf:

```
nano main.tf
```

Scroll to **bottom of your variable blocks**, then add:

```
variable "tags" {  
    type = map(string)  
}
```

```
output "tags" {  
    value = var.tags  
}
```



```
GNU nano 7.2 main.tf *  
default = ""  
description = "Short-lived API session token used during apply operations"  
sensitive = true  
nullable = false  
  
validation {  
    condition = can(regex("^[A-Za-z0-9-_{20,}$", var.api_session_token))  
    error_message = "The API session token must be at least 20 characters and contain only alphanumeric characters, hyphens, and underscores"  
}  
}  
  
# Task 3 variables  
variable "environment" {}  
variable "project_name" {}  
variable "primary_subnet_id" {}  
variable "subnet_count" {}  
variable "monitoring" {}  
# Task 4 variables  
variable "tags" {  
    type = map(string)  
}  
  
output "tags" {  
    value = var.tags  
}
```

Save (Ctrl+O → Enter, Ctrl+X).

◆ Step 2: Add tags values in terraform.tfvars

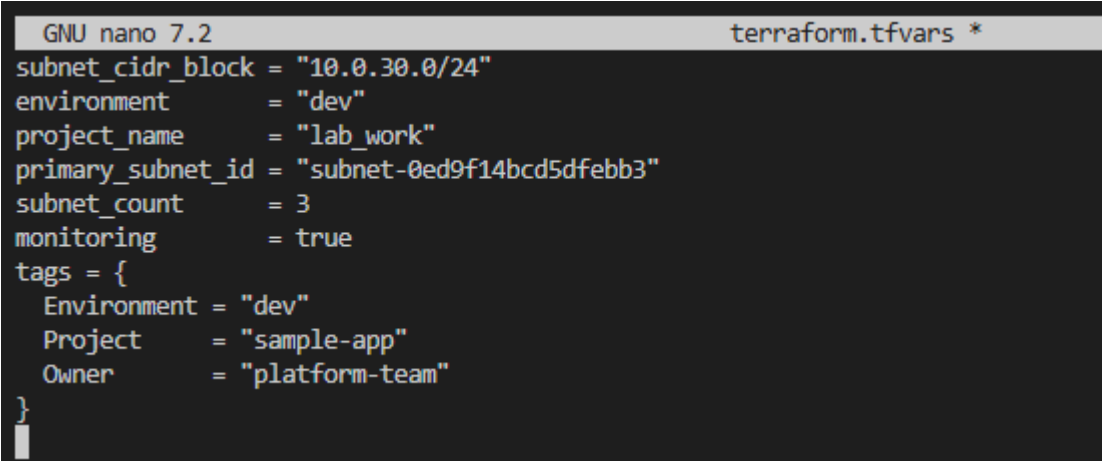
Open or create terraform.tfvars:

```
nano terraform.tfvars
```

Add (or append) the following:

```
tags = {  
    Environment = "dev"  
    Project     = "sample-app"  
    Owner       = "platform-team"  
}
```

Save (Ctrl+O → Enter, Ctrl+X).

A screenshot of a terminal window with a dark background. The title bar at the top shows "GNU nano 7.2" on the left and "terraform.tfvars *" on the right. The terminal displays the following text:

```
subnet_cidr_block = "10.0.30.0/24"  
environment       = "dev"  
project_name      = "lab_work"  
primary_subnet_id = "subnet-0ed9f14bcd5dfebb3"  
subnet_count      = 3  
monitoring        = true  
tags = {  
    Environment = "dev"  
    Project     = "sample-app"  
    Owner       = "platform-team"  
}
```

A white cursor is visible at the end of the closing curly brace on the last line.

◆ Step 3: Apply Terraform to check tags

```
terraform apply -auto-approve
```

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ terraform apply -auto-approve \
-var "api_session_token=my_API_session_Token_12345"
```

Changes to Outputs:

```
+ tags = {
+   Environment = "dev"
+   Owner       = "platform-team"
+   Project      = "sample-app"
+ }
```

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

```
api_session_token_output = <sensitive>
is_production = false
monitoring_enabled = true
primary_public_subnet = "subnet-0ed9f14bcd5dfebb3"
resource_name = "lab_work-dev"
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner"       = "platform-team"
  "Project"     = "sample-app"
})
```

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

◆ Step 4: Add server_config object variable in main.tf

Open main.tf:

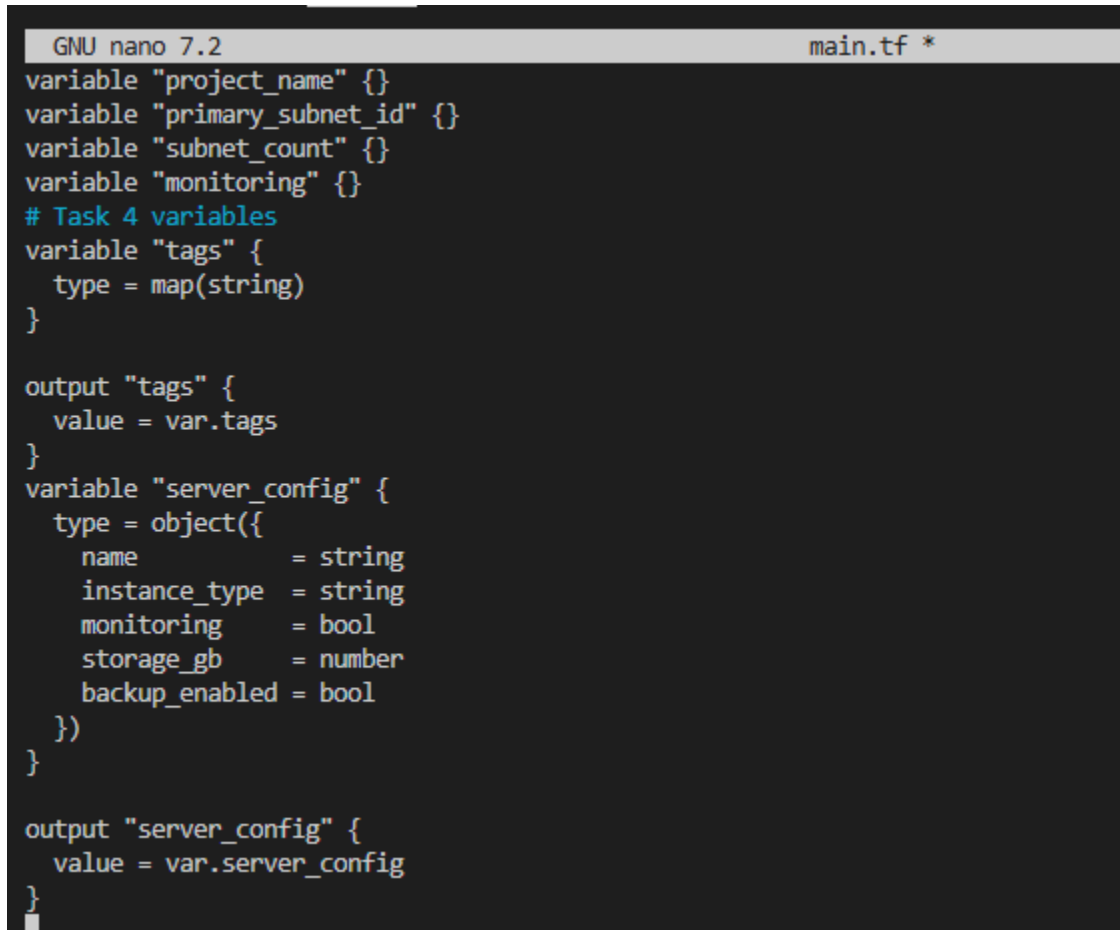
```
nano main.tf
```

Scroll to **bottom of variable blocks**, then add:

```
variable "server_config" {
  type = object({
    name          = string
    instance_type = string
    monitoring     = bool
    storage_gb     = number
    backup_enabled = bool
  })
}
```

```
output "server_config" {  
    value = var.server_config  
}
```

Save (Ctrl+O → Enter, Ctrl+X).

A screenshot of a terminal window with a dark background. The title bar at the top shows "GNU nano 7.2" on the left and "main.tf *" on the right. The terminal contains Terraform code. It starts with several variable declarations: "project_name", "primary_subnet_id", "subnet_count", and "monitoring", each followed by an empty object {}. Then there is a comment "# Task 4 variables". This is followed by a "tags" variable declaration with a "type = map(string)" assignment. Next is an "output 'tags'" block with "value = var.tags". Then a "server_config" variable declaration with a "type = object({ ... })" block containing five attributes: "name" (string), "instance_type" (string), "monitoring" (bool), "storage_gb" (number), and "backup_enabled" (bool). Finally, there is an "output 'server_config'" block with "value = var.server_config".

```
GNU nano 7.2                                     main.tf *  
variable "project_name" {}  
variable "primary_subnet_id" {}  
variable "subnet_count" {}  
variable "monitoring" {}  
# Task 4 variables  
variable "tags" {  
    type = map(string)  
}  
  
output "tags" {  
    value = var.tags  
}  
  
variable "server_config" {  
    type = object({  
        name          = string  
        instance_type = string  
        monitoring    = bool  
        storage_gb    = number  
        backup_enabled = bool  
    })  
}  
  
output "server_config" {  
    value = var.server_config  
}
```

◆ Step 5: Add server_config values in terraform.tfvars

Open terraform.tfvars:

```
nano terraform.tfvars
```

Add (or append):

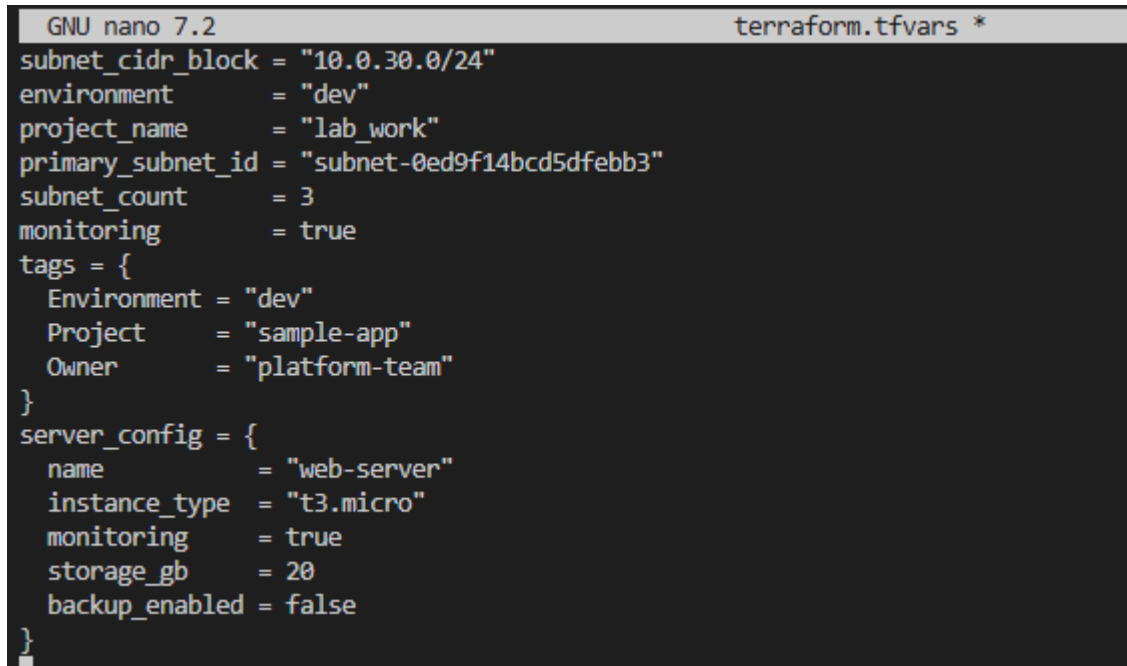
```
server_config = {  
    name          = "web-server"  
    instance_type = "t3.micro"  
    monitoring    = true
```



```
storage_gb    = 20

backup_enabled = false
}
```

Save (Ctrl+O → Enter, Ctrl+X).

A screenshot of a terminal window with a dark background. The title bar at the top shows 'GNU nano 7.2' on the left and 'terraform.tfvars *' on the right. The terminal displays the following Terraform configuration code:

```
subnet_cidr_block = "10.0.30.0/24"
environment       = "dev"
project_name      = "lab_work"
primary_subnet_id = "subnet-0ed9f14bcd5dfebb3"
subnet_count      = 3
monitoring        = true
tags = {
  Environment = "dev"
  Project     = "sample-app"
  Owner       = "platform-team"
}
server_config = {
  name           = "web-server"
  instance_type  = "t3.micro"
  monitoring     = true
  storage_gb     = 20
  backup_enabled = false
}
```

◆ Step 6: Apply Terraform to check server_config

terraform apply -auto-approve

- Output will show:

```
server_config = {
  "name" = "web-server"
  "instance_type" = "t3.micro"
  "monitoring" = true
  "storage_gb" = 20
  "backup_enabled" = false
}
```

```
+ storage_gb    = 20
}
```

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

```
api_session_token_output = <sensitive>
is_production = false
monitoring_enabled = true
primary_public_subnet = "subnet-0ed9f14bcd5dfebb3"
resource_name = "lab_work-dev"
server_config = {
  "backup_enabled" = false
  "instance_type" = "t3.micro"
  "monitoring" = true
  "name" = "web-server"
  "storage_gb" = 20
}
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

Task 5 — Collections: list, tuple, set & mutation via locals

Step 1 — Define collection variables in main.tf

Create or edit your **main.tf** and define the variables as described:

```
variable "server_names" {
  type    = list(string)

  default = ["web-2", "web-1", "web-2"] # duplicates allowed, order preserved
}
```

```
variable "server_metadata" {
  type    = tuple([string, number, bool])

  default = ["web-1", 4, true] # tuple: fixed size, mixed types allowed
}
```

```
variable "availability_zones" {
```

```

    type    = set(string)

    default = ["me-central-1b", "me-central-1a", "me-central-1b"] # set: no duplicates, order not
guaranteed
}

output "compare_collections" {
    value = {

        list_example = var.server_names

        tuple_example = var.server_metadata

        set_example  = var.availability_zones

    }
}

```

```

GNU nano 7.2                                main.tf *
value = local.monitoring_enabled
}
# task 5
variable "server_names" {
    type    = list(string)
    default = ["web-2", "web-1", "web-2"] # duplicates allowed, order preserved
}

variable "server_metadata" {
    type    = tuple([string, number, bool])
    default = ["web-1", 4, true] # tuple: fixed size, mixed types allowed
}

variable "availability_zones" {
    type    = set(string)
    default = ["me-central-1b", "me-central-1a", "me-central-1b"] # set: no duplicates, order not guaranteed
}

output "compare_collections" {
    value = {
        list_example = var.server_names
        tuple_example = var.server_metadata
        set_example  = var.availability_zones
    }
}

```

Step 2 — Apply Terraform and check output

Run these commands in your Codespace terminal: terraform init

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ terraform init

terraform apply -auto-approve -var "api_session_token=my_API_session_Token_12345"
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.28.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

Changes to Outputs:
+ compare_collections      = {
+   list_example = [
+     "web-2",
+     "web-1",
+     "web-2",
+   ]
+   set_example  = [
+     "me-central-1a",
+     "me-central-1b",
+   ]
+   tuple example = [
```

Step 3 — Create locals.tf to demonstrate mutation

Create a **locals.tf** file (or add to it) and define mutations:

```
locals {

    mutated_list = setunion(var.server_names, ["web-3"])    # adds "web-3" to list, returns a set

    mutated_tuple = setunion(var.server_metadata, ["web-2"]) # tuple converted to set, new
element added

    mutated_set = setunion(var.availability_zones, ["me-central-1c"]) # adds new AZ

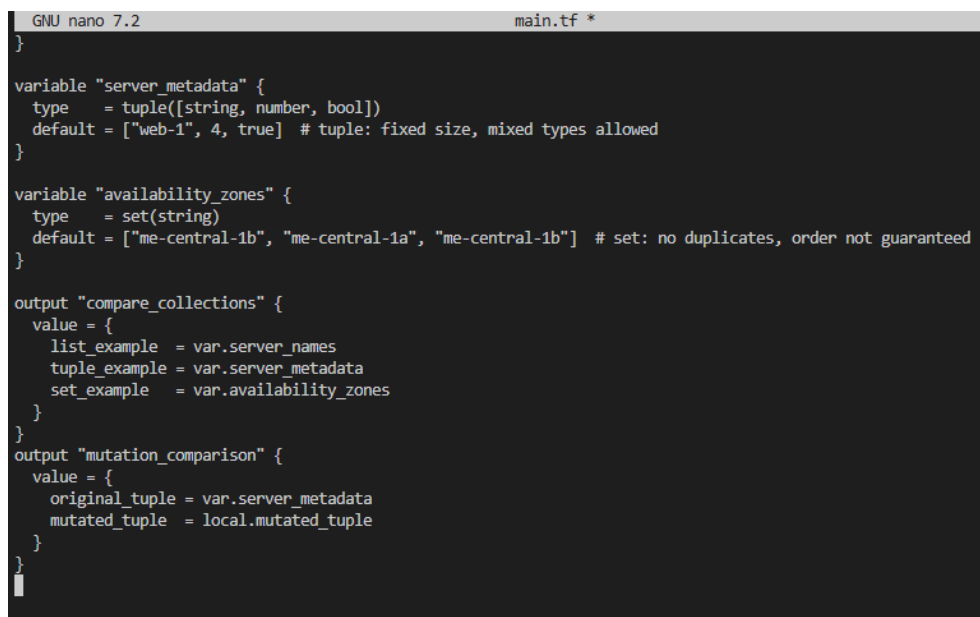
}
```

```
GNU nano 7.2                                locals.tf *
locals {
  resource_name      = "${var.project_name}-${var.environment}"
  primary_public_subnet = var.primary_subnet_id
  subnet_count       = var.subnet_count
  is_production       = var.environment == "prod"
  monitoring_enabled  = var.monitoring || local.is_production
}
locals {
  mutated_list = setunion(var.server_names, ["web-3"])    # adds "web-3" to list, returns a set
  mutated_tuple = setunion(var.server_metadata, ["web-2"]) # tuple converted to set, new element added
  mutated_set   = setunion(var.availability_zones, ["me-central-1c"]) # adds new AZ
}
```

Step 4 — Add mutation comparison output

Back in main.tf, add an output block to compare original vs mutated tuple:

```
output "mutation_comparison" {  
  value = {  
    original_tuple = var.server_metadata  
    mutated_tuple  = local.mutated_tuple  
  }  
}
```



```
GNU nano 7.2 main.tf *  
}  
  
variable "server_metadata" {  
  type = tuple([string, number, bool])  
  default = ["web-1", 4, true] # tuple: fixed size, mixed types allowed  
}  
  
variable "availability_zones" {  
  type = set(string)  
  default = ["me-central-1b", "me-central-1a", "me-central-1b"] # set: no duplicates, order not guaranteed  
}  
  
output "compare_collections" {  
  value = {  
    list_example = var.server_names  
    tuple_example = var.server_metadata  
    set_example = var.availability_zones  
  }  
}  
output "mutation_comparison" {  
  value = {  
    original_tuple = var.server_metadata  
    mutated_tuple = local.mutated_tuple  
  }  
}
```

- Apply again: terraform apply -auto-approve

```

@23-22411-007-oss → /workspaces/Lab-11 (main) $ terraform apply -auto-approve -var "api_session_token=my_API_session_token_12345"

Changes to Outputs:
  + mutation_comparison = {
    + mutated_tuple = [
      + "4",
      + "true",
      + "web-1",
      + "web-2",
    ]
    + original_tuple = [
      + "web-1",
      + 4,
      + true,
    ]
  }

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:
api_session_token_output = <sensitive>
compare_collections = {
  "list_example" = tolist([
    "web-2",
    "web-1",
  ])
}

mutation_comparison = {
  "mutated_tuple" = toset([
    "4",
    "true",
    "web-1",
    "web-2",
  ])
  "original_tuple" = [
    "web-1",
    4,
    true,
  ]
}
primary_public_subnet = "subnet-0ed9f14bcd5dfebb3"
resource_name = "lab_work-dev"
server_config = {
  "backup_enabled" = false
  "instance_type" = "t3.micro"
  "monitoring" = true
  "name" = "web-server"
  "storage_gb" = 20
}
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})
@23-22411-007-oss → /workspaces/Lab-11 (main) $

```

Task 6 — Null, any type & dynamic values

Step 1 — Define a Null variable

In **main.tf**, define the optional_tag variable:

```

variable "optional_tag" {
  type      = string
  description = "A tag that may or may not be provided"
  default   = null
}

```

```
# task 6
variable "optional_tag" {
  type      = string
  description = "A tag that may or may not be provided"
  default   = null
}
```

Step 2 — Merge tags using locals

In **locals.tf**, create a merged map of tags, including the optional tag only if it is not null:

```
locals {
  server_tags = merge(
    { Name = "web-server" },
    var.optional_tag != null ? { Custom = var.optional_tag } : {}
  )
}
```

```
GNU nano 7.2      locals.tf *
locals {
  resource_name      = "${var.project_name}-${var.environment}"
  primary_public_subnet = var.primary_subnet_id
  subnet_count       = var.subnet_count
  is_production       = var.environment == "prod"
  monitoring_enabled  = var.monitoring || local.is_production
}
locals {
  mutated_list = setunion(var.server_names, ["web-3"]) # adds "web-3" to list, returns a set
  mutated_tuple = setunion(var.server_metadata, ["web-2"]) # tuple converted to set, new element added
  mutated_set = setunion(var.availability_zones, ["me-central-1c"]) # adds new AZ
}
# task 6
locals {
  server_tags = merge(
    { Name = "web-server" },
    var.optional_tag != null ? { Custom = var.optional_tag } : {}
  )
}
```

Step 3 — Output merged tags

Add in **main.tf**:

```
output "optional_tag" {
  value = local.server_tags
}
```

```
# task 6
variable "optional_tag" {
  type      = string
  description = "A tag that may or may not be provided"
  default    = null
}

output "optional_tag" {
  value = local.server_tags
}
```

Step 4 — Test Null variable (no value set)

1. **Do NOT** set `optional_tag` in `terraform.tfvars`.

2. Run:

```
terraform apply -auto-approve
```

3. Observe output:

```
optional_tag = {
```

```
  Name = "web-server"
```

```
}
```



```

@23-22411-007-oss → /workspaces/lab-11 (main) $ terraform apply -auto-approve -var "api_session_token=eyJ0I_
sion_token_12345"

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no
changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

api_session_token output → example
compare collections = {
  "list_example" = list([
    "web-2",
    "web-1",
    "web-2",
  ])
  "set_example" = toset([
    "aws-central-tx",
    "aws-central-us",
  ])
  "tuple_example" = [
    "web-1",
    4,
    true,
  ]
}
is_production = false

```

```

    "true",
    "web-1",
    "web-2",
  ])
  "original_tuple" = [
    "web-1",
    4,
    true,
  ]
}
optional_tag = {
  "Name" = "web-server"
}
primary_public_subnet = "subnet-0ed9f14bcd5df6bb"
resource_name = "lab_work-dev"
server_config = {
  "backup_enabled" = false
  "instance_type" = "t3.micro"
  "monitoring" = true
  "name" = "web-server"
  "storage_gb" = 20
}
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})

```

Step 5 — Test Null variable (with value set)

1. Add to **terraform.tfvars**:

`optional_tag = "dev"`

```

GNU nano 7.2                                terraform.tfvars *
subnet_cidr_block = "10.0.30.0/24"
environment       = "dev"
project_name      = "lab_work"
primary_subnet_id = "subnet-0ed9f14bcd5dfebb3"
subnet_count      = 3
monitoring        = true
tags = {
  Environment = "dev"
  Project     = "sample-app"
  Owner       = "platform-team"
}
server_config = {
  name          = "web-server"
  instance_type = "t3.micro"
  monitoring     = true
  storage_gb     = 20
  backup_enabled = false
}
optional_tag = "dev"

```

2. Run: `terraform apply -auto-approve`

```

    "web-1",
    "web-2",
  ])
  "original_tuple" = [
    "web-1",
    4,
    true,
  ]
}
optional_tag = {
  "Custom" = "dev"
  "Name"   = "web-server"
}
primary_public_subnet = "subnet-0ed9f14bcd5dfebb3"
resource_name = "lab_work-dev"
server_config = {
  "backup_enabled" = false
  "instance_type"  = "t3.micro"
  "monitoring"     = true
  "name"           = "web-server"
  "storage_gb"     = 20
}
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner"       = "platform-team"
  "Project"     = "sample-app"
})

```

Step 6 — Any type variable

In **main.tf**, define a variable that can accept **any type**:

```
variable "dynamic_value" {
```

```

type      = any

description = "A variable that can accept any data type"

default    = null
}

output "value_received" {
  value = var.dynamic_value
}

```

```

# task 6
variable "optional_tag" {
  type      = string
  description = "A tag that may or may not be provided"
  default    = null
}

output "optional_tag" {
  value = local.server_tags
}

variable "dynamic_value" {
  type      = any
  description = "A variable that can accept any data type"
  default    = null
}

output "value_received" {
  value = var.dynamic_value
}

```

Step 7 — Test dynamic values

For each type, update **terraform.tfvars** and run `terraform apply -auto-approve`. Capture screenshots.

a) String

```
dynamic_value = "hello"
```

```

    }
    primary_public_subnet = "subnet-0ed9f14bcd5dfebb3"
    resource_name = "lab_work-dev"
    server_config = {
        "backup_enabled" = false
        "instance_type" = "t3.micro"
        "monitoring" = true
        "name" = "web-server"
        "storage_gb" = 20
    }
    subnet_cidr_block_output = "10.0.30.0/24"
    subnet_count = 3
    tags = tomap({
        "Environment" = "dev"
        "Owner" = "platform-team"
        "Project" = "sample-app"
    })
    value_received = "hello"
@23-22411-007-oss →/workspaces/Lab-11 (main) $

```

Output: "hello"

b) Number

dynamic_value = 42

Output: 42

```

    }
    optional_tag = {
        "Custom" = "dev"
        "Name" = "web-server"
    }
    primary_public_subnet = "subnet-0ed9f14bcd5dfebb3"
    resource_name = "lab_work-dev"
    server_config = {
        "backup_enabled" = false
        "instance_type" = "t3.micro"
        "monitoring" = true
        "name" = "web-server"
        "storage_gb" = 20
    }
    subnet_cidr_block_output = "10.0.30.0/24"
    subnet_count = 3
    tags = tomap({
        "Environment" = "dev"
        "Owner" = "platform-team"
        "Project" = "sample-app"
    })
    value_received = 42
@23-22411-007-oss →/workspaces/Lab-11 (main) $

```

c) List

dynamic_value = ["a", "b", "c"]

Output: ["a", "b", "c"]

```
]
primary_public_subnet = "subnet-0ed9f14bcd5dfebb3"
resource_name = "lab_work-dev"
server_config = {
  "backup_enabled" = false
  "instance_type" = "t3.micro"
  "monitoring" = true
  "name" = "web-server"
  "storage_gb" = 20
}
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})
value_received = [
  "a",
  "b",
  "c",
]
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

d) Map / Object

dynamic_value = {

name = "server"

cpu = 4

}

Output:

{

name = "server"

cpu = 4

}

```

primary_public_subnet = "subnet-0ed9f14bcd5dfebb3"
resource_name = "lab_work-dev"
server_config = {
  "backup_enabled" = false
  "instance_type" = "t3.micro"
  "monitoring" = true
  "name" = "web-server"
  "storage_gb" = 20
}
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})
value_received = {
  "cpu" = 4
  "name" = "server"
}
@23-22411-007-oss →/workspaces/Lab-11 (main) $

```

e) Null

dynamic_value = null

Output: null

```

4,
true,
]
}
optional_tag = {
  "Custom" = "dev"
  "Name" = "web-server"
}
primary_public_subnet = "subnet-0ed9f14bcd5dfebb3"
resource_name = "lab_work-dev"
server_config = {
  "backup_enabled" = false
  "instance_type" = "t3.micro"
  "monitoring" = true
  "name" = "web-server"
  "storage_gb" = 20
}
subnet_cidr_block_output = "10.0.30.0/24"
subnet_count = 3
tags = tomap({
  "Environment" = "dev"
  "Owner" = "platform-team"
  "Project" = "sample-app"
})
@23-22411-007-oss →/workspaces/Lab-11 (main) $

```

Task 7 — Git ignore

Step 1 — Create the .gitignore file

In your project directory (Lab11), run:

```
touch .gitignore
```

- This creates an empty .gitignore file.

Step 2 — Add entries to .gitignore

Open .gitignore in an editor and add the following lines:

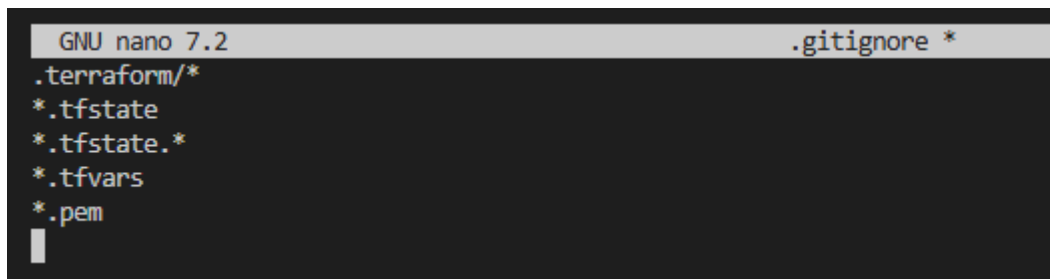
```
.terraform/*
```

```
*.tfstate
```

```
*.tfstate.*
```

```
*.tfvars
```

```
*.pem
```

A screenshot of a terminal window showing the GNU nano 7.2 editor. The file being edited is .gitignore. The contents of the file are: .terraform/*, *.tfstate, *.tfstate.*, *.tfvars, and *.pem. The cursor is at the end of the file.

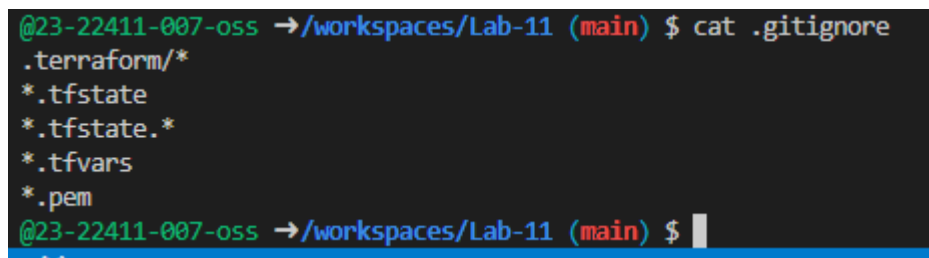
```
GNU nano 7.2 .gitignore *
.terraform/*
*.tfstate
*.tfstate.*
*.tfvars
*.pem

```

Step 3 — Save and verify

- Run this command to check the contents:

```
cat .gitignore
```

A screenshot of a terminal window showing the output of the command cat .gitignore. The output is: .terraform/*, *.tfstate, *.tfstate.*, *.tfvars, and *.pem. The prompt shows the user is in the main branch of a repository.

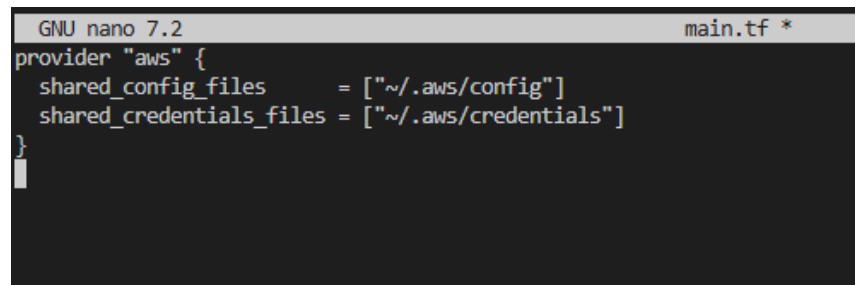
```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ cat .gitignore
.terraform/*
*.tfstate
*.tfstate.*
*.tfvars
*.pem
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

Step 1 — Clean previous files

1. Open terraform.tfvars and remove all previous values (or delete the file).
2. Open locals.tf and empty it (or delete).
3. Replace main.tf with only the **provider block**:

```
provider "aws" {
  shared_config_files = ["~/.aws/config"]
}
```

```
shared_credentials_files = ["~/.aws/credentials"]  
}
```



```
GNU nano 7.2 main.tf *  
provider "aws" {  
  shared_config_files    = ["~/.aws/config"]  
  shared_credentials_files = ["~/.aws/credentials"]  
}
```

Step 2 — Define variables

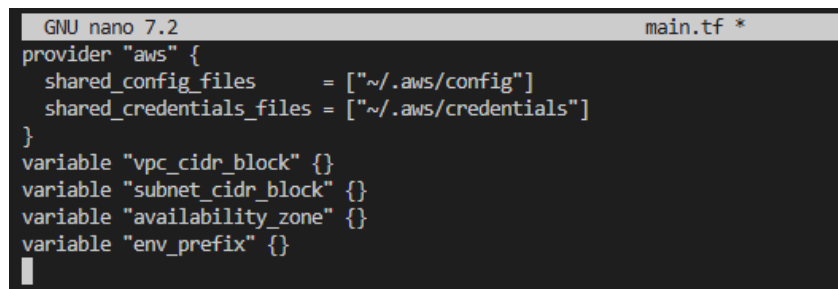
Below the provider block in main.tf, add:

```
variable "vpc_cidr_block" {}
```

```
variable "subnet_cidr_block" {}
```

```
variable "availability_zone" {}
```

```
variable "env_prefix" {}
```



```
GNU nano 7.2 main.tf *  
provider "aws" {  
  shared_config_files    = ["~/.aws/config"]  
  shared_credentials_files = ["~/.aws/credentials"]  
}  
variable "vpc_cidr_block" {}  
variable "subnet_cidr_block" {}  
variable "availability_zone" {}  
variable "env_prefix" {}
```

Step 3 — Create the VPC

Add the VPC resource in main.tf:

```
resource "aws_vpc" "myapp_vpc" {  
  cidr_block = var.vpc_cidr_block  
  tags = {  
    Name = "${var.env_prefix}-vpc"  
  }  
}
```



```

GNU nano 7.2                                main.tf *
provider "aws" {
  shared_config_files    = ["~/.aws/config"]
  shared_credentials_files = ["~/.aws/credentials"]
}
variable "vpc_cidr_block" {}
variable "subnet_cidr_block" {}
variable "availability_zone" {}
variable "env_prefix" {}
resource "aws_vpc" "myapp_vpc" {
  cidr_block = var.vpc_cidr_block
  tags = {
    Name = "${var.env_prefix}-vpc"
  }
}

```

Step 4 — Create Subnet

Add subnet resource:

```

resource "aws_subnet" "myapp_subnet_1" {

  vpc_id      = aws_vpc.myapp_vpc.id

  cidr_block   = var.subnet_cidr_block

  availability_zone = var.availability_zone

  tags = {

    Name = "${var.env_prefix}-subnet-1"

  }

}

```

```

}
resource "aws_subnet" "myapp_subnet_1" {
  vpc_id      = aws_vpc.myapp_vpc.id
  cidr_block   = var.subnet_cidr_block
  availability_zone = var.availability_zone
  tags = {
    Name = "${var.env_prefix}-subnet-1"
  }
}

```

Step 5 — Populate terraform.tfvars

Create terraform.tfvars and add:

```

vpc_cidr_block    = "10.0.0.0/16"

subnet_cidr_block = "10.0.10.0/24"

availability_zone = "me-central-1a"

```

```
env_prefix    = "dev"
```

```
GNU nano 7.2 terraform.tfvars *
vpc_cidr_block    = "10.0.0.0/16"
subnet_cidr_block = "10.0.10.0/24"
availability_zone  = "me-central-1a"
env_prefix        = "dev"
█
```

Step 6 — Apply VPC and Subnet

Run:

```
terraform init    # Initialize Terraform
```

```
terraform apply -auto-approve
```

Check AWS Console → VPCs and Subnets to verify creation.

```
- true,
]
} -> null
- optional_tag          = {
  - Custom = "dev"
  - Name   = "web-server"
} -> null
- primary_public_subnet = "subnet-0ed9f14bcd5dfebb3" -> null
- resource_name         = "lab_work-dev" -> null
- server_config          = {
  - backup_enabled = false
  - instance_type  = "t3.micro"
  - monitoring     = true
  - name           = "web-server"
  - storage_gb     = 20
} -> null
- subnet_cidr_block_output = "10.0.30.0/24" -> null
- subnet_count             = 3 -> null
- tags                     = {
  - Environment = "dev"
  - Owner       = "platform-team"
  - Project     = "sample-app"
} -> null
aws_vpc.myapp_vpc: Creating...
aws_vpc.myapp_vpc: Creation complete after 1s [id=vpc-0284549ca1fb501a9]
aws_subnet.myapp_subnet_1: Creating...
aws_subnet.myapp_subnet_1: Creation complete after 1s [id=subnet-024e873f19b987ee0]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
@23-22411-007-oss → /workspaces/Lab-11 (main) $ █
```

Step 7 — Create Internet Gateway & Custom Route Table

Add the following to main.tf **below the subnet resource**:

```
resource "aws_internet_gateway" "myapp_igw" {
  vpc_id = aws_vpc.myapp_vpc.id
```

```
tags = {  
    Name = "${var.env_prefix}-igw"  
}  
}
```

```
resource "aws_route_table" "myapp_route_table" {  
    vpc_id = aws_vpc.myapp_vpc.id
```

```
    route {  
        cidr_block = "0.0.0.0/0"  
        gateway_id = aws_internet_gateway.myapp_igw.id  
    }
```

```
tags = {  
    Name = "${var.env_prefix}-rt"  
}  
}
```

```
GNU nano 7.2                                     main.tf *
cidr_block      = var.subnet_cidr_block
availability_zone = var.availability_zone
tags = {
    Name = "${var.env_prefix}-subnet-1"
}
}
resource "aws_internet_gateway" "myapp_igw" {
    vpc_id = aws_vpc.myapp_vpc.id
    tags = {
        Name = "${var.env_prefix}-igw"
    }
}

resource "aws_route_table" "myapp_route_table" {
    vpc_id = aws_vpc.myapp_vpc.id

    route {
        cidr_block = "0.0.0.0/0"
        gateway_id = aws_internet_gateway.myapp_igw.id
    }

    tags = {
        Name = "${var.env_prefix}-rt"
    }
}
```

Apply changes: terraform apply -auto-approve

```

+ resource "aws_route_table" "myapp_route_table" {
  + arn                = (known after apply)
  + id                 = (known after apply)
  + owner_id           = (known after apply)
  + propagating_vgws   = (known after apply)
  + region              = "me-central-1"
  + route              = [
    + {
      + cidr_block        = "0.0.0.0/0"
      + gateway_id        = (known after apply)
      # (11 unchanged attributes hidden)
    },
  ]
  + tags               = {
    + "Name" = "dev-rt"
  }
  + tags_all           = {
    + "Name" = "dev-rt"
  }
  + vpc_id             = "vpc-0284549ca1fb501a9"
}

```

Plan: 2 to add, 0 to change, 0 to destroy.

aws_internet_gateway.myapp_igw: Creating...

aws_internet_gateway.myapp_igw: Creation complete after 0s [id=igw-0f2d579e8e5d12ab9]

aws_route_table.myapp_route_table: Creating...

aws_route_table.myapp_route_table: Creation complete after 2s [id=rtb-0cd8e60196c956420]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

@23-22411-007-oss →/workspaces/Lab-11 (main) \$

Step 8 — Associate Route Table with Subnet

Add:

```

resource "aws_route_table_association" "a_rtb_subnet" {
  subnet_id    = aws_subnet.myapp_subnet_1.id
  route_table_id = aws_route_table.myapp_route_table.id
}

```

```

}
resource "aws_route_table_association" "a_rtb_subnet" {
  subnet_id    = aws_subnet.myapp_subnet_1.id
  route_table_id = aws_route_table.myapp_route_table.id
}

```

Apply: terraform apply -auto-approve

```

@23-22411-007-oss →/workspaces/Lab-11 (main) $ terraform apply -auto-approve
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-0284549ca1fb501a9]
aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-0f2d579e8e5d12ab9]
aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-024e873f19b987ee0]
aws_route_table.myapp_route_table: Refreshing state... [id=rtb-0cd8e60196c956420]
aws_route_table_association.a_rtb_subnet: Refreshing state... [id=rtbassoc-0109feca9456a994d]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no
changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
@23-22411-007-oss →/workspaces/Lab-11 (main) $

```

Step 9 — Switch to Default Route Table

1. Remove or comment out:

```

resource "aws_route_table" "myapp_route_table" {...}

resource "aws_route_table_association" "a_rtb_subnet" {...}

```

2. Add:

```

resource "aws_default_route_table" "main_rt" {

  default_route_table_id = aws_vpc.myapp_vpc.default_route_table_id

  route {

    cidr_block = "0.0.0.0/0"

    gateway_id = aws_internet_gateway.myapp_igw.id
  }

  tags = {

    Name = "${var.env_prefix}-rt"
  }
}

```

```
GNU nano 7.2                                main.tf *
}
resource "aws_internet_gateway" "myapp_igw" {
  vpc_id = aws_vpc.myapp_vpc.id
  tags = {
    Name = "${var.env_prefix}-igw"
  }
}

resource "aws_route_table" "myapp_route_table" {
  vpc_id = aws_vpc.myapp_vpc.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.myapp_igw.id
  }

  tags = {
    Name = "${var.env_prefix}-rt"
  }
}

resource "aws_route_table_association" "a_rtb_subnet" {
  subnet_id      = aws_subnet.myapp_subnet_1.id
  route_table_id = aws_route_table.myapp_route_table.id
}
|
```

Apply changes:

terraform apply -auto-approve

Verify in AWS Console → VPC → Route Tables → Default has 0.0.0.0/0 → IGW.

```

@23-22411-007-oss → /workspaces/Lab-11 (main) $ terraform apply -auto-approve
aws_vpc.myapp_vpc: Refreshing state... [id=vpc-0284549ca1fb501a9]
aws_internet_gateway.myapp_igw: Refreshing state... [id=igw-0f2d579e8e5d12ab9]
aws_subnet.myapp_subnet_1: Refreshing state... [id=subnet-024e873f19b987ee0]
aws_route_table.myapp_route_table: Refreshing state... [id=rtb-0cd8e60196c956420]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated
with the following symbols:
  + create

Terraform will perform the following actions:

# aws_route_table_association.a_rtb_subnet will be created
+ resource "aws_route_table_association" "a_rtb_subnet" {
  + id            = (known after apply)
  + region        = "me-central-1"
  + route_table_id = "rtb-0cd8e60196c956420"
  + subnet_id     = "subnet-024e873f19b987ee0"
}

Plan: 1 to add, 0 to change, 0 to destroy.
aws_route_table_association.a_rtb_subnet: Creating...
aws_route_table_association.a_rtb_subnet: Creation complete after 0s [id=rtbassoc-0109feca9456a994d]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
@23-22411-007-oss → /workspaces/Lab-11 (main) $

```

Task 9 — Security Group, Key Pair, EC2 Instance, user_data & nginx

Step 1 — Add my_ip variable

File: main.tf

Add below your existing variables:

variable "my_ip" {}

```

}
# task 9
variable "my_ip" {}

```

Step 2 — Get your public IP and set terraform.tfvars

From Codespace shell:

curl icanhazip.com

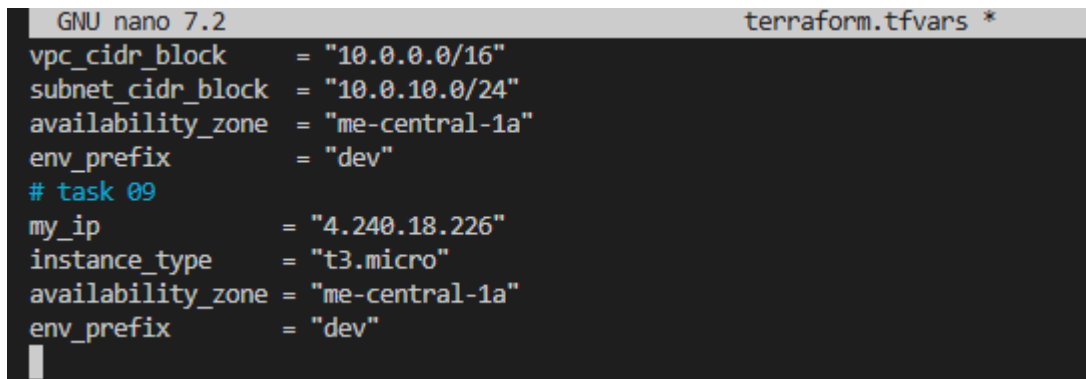
```

@23-22411-007-oss → /workspaces/Lab-11 (main) $ curl icanhazip.com
4.240.18.226
@23-22411-007-oss → /workspaces/Lab-11 (main) $

```

- Copy your IP, 4.240.18.226
- Edit terraform.tfvars:


```
my_ip      = "4.240.18.226"
instance_type  = "t3.micro"
availability_zone = "me-central-1a"
env_prefix   = "dev"
```



```
GNU nano 7.2 terraform.tfvars *
vpc_cidr_block = "10.0.0.0/16"
subnet_cidr_block = "10.0.10.0/24"
availability_zone = "me-central-1a"
env_prefix      = "dev"
# task 09
my_ip           = "4.240.18.226"
instance_type   = "t3.micro"
availability_zone = "me-central-1a"
env_prefix      = "dev"
```

Step 3 — Create Security Group

File: main.tf

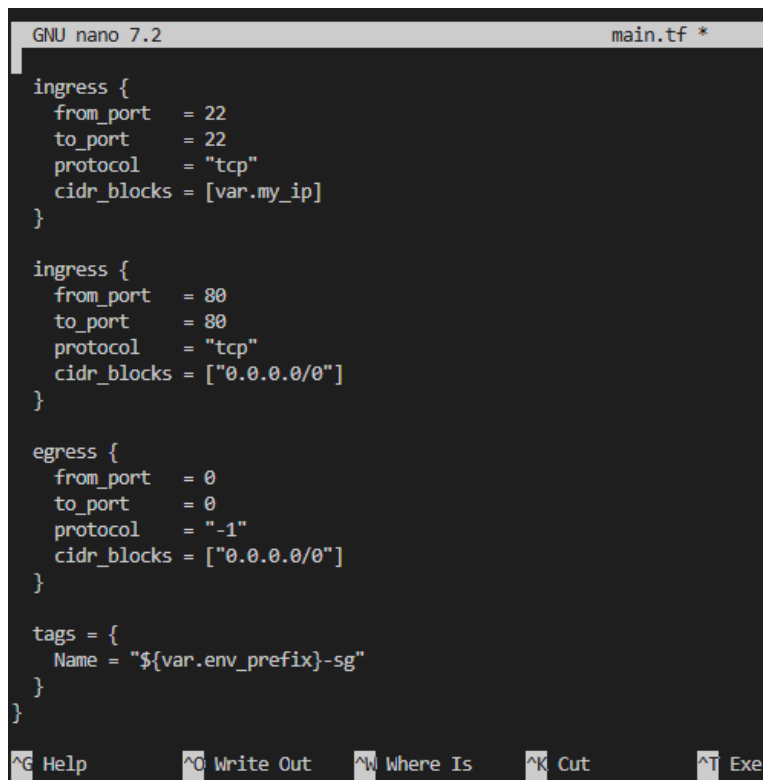
Add:

```
resource "aws_default_security_group" "myapp_sg" {
  vpc_id = aws_vpc.myapp_vpc.id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = [var.my_ip]
  }

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

```
}  
  
egress {  
  from_port = 0  
  to_port   = 0  
  protocol  = "-1"  
  cidr_blocks = ["0.0.0.0/0"]  
}  
  
tags = {  
  Name = "${var.env_prefix}-sg"  
}  
}
```



```
GNU nano 7.2 main.tf *  
  
ingress {  
  from_port = 22  
  to_port   = 22  
  protocol  = "tcp"  
  cidr_blocks = [var.my_ip]  
}  
  
ingress {  
  from_port = 80  
  to_port   = 80  
  protocol  = "tcp"  
  cidr_blocks = ["0.0.0.0/0"]  
}  
  
egress {  
  from_port = 0  
  to_port   = 0  
  protocol  = "-1"  
  cidr_blocks = ["0.0.0.0/0"]  
}  
  
tags = {  
  Name = "${var.env_prefix}-sg"  
}  
}
```

Apply: terraform apply -auto-approve

```

+ name           = (known after apply)
+ name_prefix    = (known after apply)
+ owner_id       = (known after apply)
+ region         = "me-central-1"
+ revoke_rules_on_delete = false
+ tags           = {
  + "Name" = "dev-sg"
}
+ tags_all       = {
  + "Name" = "dev-sg"
}
+ vpc_id         = "vpc-0284549ca1fb501a9"
}

```

Plan: 1 to add, 0 to change, 0 to destroy.

aws_default_security_group.myapp_sg: Creating...

aws_default_security_group.myapp_sg: Creation complete after 2s [id=sg-016a544d965355413]

Warning: Value for undeclared variable

The root module does not declare a variable named "instance_type" but a value was found in file "terraform.tfvars". If you meant to use this value, add a "variable" block to the configuration.

To silence these warnings, use TF_VAR... environment variables to provide certain "global" settings to all configurations in your organization. To reduce the verbosity of these warnings, use the -compact-warnings option.

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

@23-22411-007-oss →/workspaces/Lab-11 (main) \$

Step 4 — Create AWS Key Pair

Run:

```
aws ec2 create-key-pair \
```

```
--key-name MyED25519Key \
```

```
--key-type ed25519 \
```

```
--key-format pem \
```

```
--query 'KeyMaterial' \
```

```
--output text > MyED25519Key.pem
```

```
chmod 600 MyED25519Key.pem
```

```

@23-22411-007-oss →/workspaces/Lab-11 (main) $ aws ec2 create-key-pair \
--key-name MyED25519Key \
--key-type ed25519 \
--key-format pem \
--query 'KeyMaterial' \
--output text > MyED25519Key.pem

chmod 600 MyED25519Key.pem
@23-22411-007-oss →/workspaces/Lab-11 (main) $

```

Add to .gitignore:

*.pem

Step 5 — Launch EC2 Instance (initial)

File: main.tf

Add EC2 resource (replace `aws_default_security_group.default_sg.id` with your SG ID if needed):

```
resource "aws_instance" "myapp-server" {  
  ami           = "ami-05524d6658fcf35b6" # Amazon Linux 2023  
  instance_type = var.instance_type  
  subnet_id     = aws_subnet.myapp_subnet_1.id  
  security_groups = [aws_default_security_group.myapp_sg.id]  
  availability_zone = var.availability_zone  
  associate_public_ip_address = true  
  key_name       = "MyED25519Key"  
  
  tags = {  
    Name = "${var.env_prefix}-ec2-instance"  
  }  
}
```



```
output "aws_instance_public_ip" {  
  value = aws_instance.myapp-server.public_ip  
}
```

```

}
# lanch ec2 instance
resource "aws_instance" "myapp-server" {
  ami           = "ami-05524d6658fcf35b6" # Amazon Linux 2023
  instance_type = var.instance_type
  subnet_id     = aws_subnet.myapp_subnet_1.id
  security_groups = sg-0fda751d4ca471641
  availability_zone = var.availability_zone
  associate_public_ip_address = true
  key_name       = "MyED25519Key"

  tags = {
    Name = "${var.env_prefix}-ec2-instance"
  }
}

output "aws_instance_public_ip" {
  value = aws_instance.myapp-server.public_ip
}

```

Apply: terraform apply -auto-approve

```

@23-22411-007-oss → /workspaces/Lab-11 (main) $ terraform apply -auto-approve
# (because aws_route_table_association.a_rtb_subnet is not in configuration)
- resource "aws_route_table_association" "a_rtb_subnet" {
  - id           = "rtbassoc-0109feca9456a994d" -> null
  - region      = "me-central-1" -> null
  - route_table_id = "rtb-0cd8e60196c956420" -> null
  - subnet_id    = "subnet-024e873f19b987ee0" -> null
  # (1 unchanged attribute hidden)
}

Plan: 1 to add, 0 to change, 3 to destroy.

Changes to Outputs:
  + aws_instance_public_ip = (known after apply)
aws_route_table_association.a_rtb_subnet: Destroying... [id=rtbassoc-0109feca9456a994d]
aws_instance.myapp-server: Creating...
aws_route_table_association.a_rtb_subnet: Destruction complete after 1s
aws_route_table.myapp_route_table: Destroying... [id=rtb-0cd8e60196c956420]
aws_route_table.myapp_route_table: Destruction complete after 0s
aws_internet_gateway.myapp_igw: Destroying... [id=igw-0f2d579e8e5d12ab9]
aws_internet_gateway.myapp_igw: Destruction complete after 1s
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 13s [id=i-007f72ea100893747]

Apply complete! Resources: 1 added, 0 changed, 3 destroyed.

Outputs:
aws_instance_public_ip = "51.112.228.27"
@23-22411-007-oss → /workspaces/Lab-11 (main) $

```

Step 6 — SSH into EC2

From Codespace shell:

```
ssh -i MyED25519Key.pem ec2-user@ 4.240.18.226
```



```
#!/bin/bash
```

```
yum update -y
```

```
yum install -y nginx
```

```
systemctl start nginx
```

```
systemctl enable nginx
```

```
EOF
```

Apply: terraform apply -auto-approve

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ terraform apply -auto-approve
- tags          = {} -> null
- tags_all      = {} -> null
- throughput    = 125 -> null
- volume_id     = "vol-04adea10662b8d4d8" -> null
- volume_size   = 8 -> null
- volume_type   = "gp3" -> null
  # (1 unchanged attribute hidden)
}
}

Plan: 1 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  ~ aws_instance_public_ip = "51.112.230.214" -> (known after apply)
aws_instance.myapp-server: Destroying... [id=i-032321c295e18d1be]
aws_instance.myapp-server: Still destroying... [id=i-032321c295e18d1be, 00m10s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-032321c295e18d1be, 00m20s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-032321c295e18d1be, 00m30s elapsed]
aws_instance.myapp-server: Destruction complete after 31s
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 13s [id=i-08eea60a3c612cab9]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:
aws_instance_public_ip = "51.112.46.162"
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

SSH in and test: curl localhost


```
[ec2-user@ip-10-0-10-125 ~]$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
[ec2-user@ip-10-0-10-125 ~]$
```

Open `http://<public-ip>` in browser.



Step 10 — Use external script for user_data

1. Create entry-script.sh:

```
cat > entry-script.sh <<'EOF'
```

```
#!/bin/bash
```

```
yum update -y
```

```
yum install -y nginx
```

```
systemctl start nginx
```

systemctl enable nginx

EOF

```
connection to 51.112.46.162 closed
@23-22411-007-oss →/workspaces/Lab-11 (main) $ cat > entry-script.sh <<'EOF'
#!/bin/bash
yum update -y
yum install -y nginx
systemctl start nginx
systemctl enable nginx
EOF
@23-22411-007-oss →/workspaces/Lab-11 (main) $ chmod +x entry-script.sh
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

2. Update EC2 resource:

user_data = file("entry-script.sh")

Apply: terraform apply -auto-approve

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ terraform apply -auto-approve
- throughput          = 125 -> null
- volume_id           = "vol-01667e25148a7089b" -> null
- volume_size         = 8 -> null
- volume_type         = "gp3" -> null
  # (1 unchanged attribute hidden)
}
}

Plan: 1 to add, 0 to change, 1 to destroy.

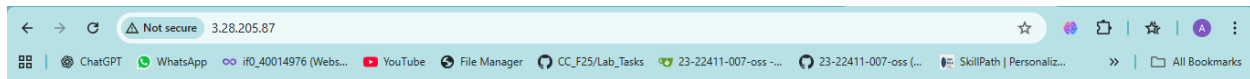
Changes to Outputs:
  ~ aws_instance_public_ip = "51.112.46.162" -> (known after apply)
aws_instance.myapp-server: Destroying... [id=i-08eea60a3c612cab9]
aws_instance.myapp-server: Still destroying... [id=i-08eea60a3c612cab9, 00m10s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-08eea60a3c612cab9, 00m20s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-08eea60a3c612cab9, 00m30s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-08eea60a3c612cab9, 00m40s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-08eea60a3c612cab9, 00m50s elapsed]
aws_instance.myapp-server: Destruction complete after 51s
aws_instance.myapp-server: Creating...
aws_instance.myapp-server: Still creating... [00m10s elapsed]
aws_instance.myapp-server: Creation complete after 14s [id=i-0812bcc72dd7f4af9]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:

aws_instance_public_ip = "3.28.205.87"
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

Reload browser to confirm nginx page.



Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Cleanup

Destroy Terraform-managed resources

If you used Terraform for Lab 11:

terraform destroy -auto-approve

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ terraform destroy -auto-approve
aws_default_route_table.main_rt: Destruction complete after 0s
aws_instance.myapp-server: Destroying... [id=i-0812bcc72dd7f4af9]
aws_internet_gateway.myapp_igw: Destroying... [id=igw-02ca1da0c491eb59e]
aws_instance.myapp-server: Still destroying... [id=i-0812bcc72dd7f4af9, 00m10s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-02ca1da0c491eb59e, 00m10s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0812bcc72dd7f4af9, 00m20s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-02ca1da0c491eb59e, 00m20s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0812bcc72dd7f4af9, 00m30s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-02ca1da0c491eb59e, 00m30s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0812bcc72dd7f4af9, 00m40s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-02ca1da0c491eb59e, 00m40s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0812bcc72dd7f4af9, 00m50s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-02ca1da0c491eb59e, 00m50s elapsed]
aws_instance.myapp-server: Still destroying... [id=i-0812bcc72dd7f4af9, 01m00s elapsed]
aws_internet_gateway.myapp_igw: Still destroying... [id=igw-02ca1da0c491eb59e, 01m00s elapsed]
aws_internet_gateway.myapp_igw: Destruction complete after 1m8s
aws_instance.myapp-server: Still destroying... [id=i-0812bcc72dd7f4af9, 01m10s elapsed]
aws_instance.myapp-server: Destruction complete after 1m11s
aws_key_pair.ssh key: Destroying... [id=serverkey]
aws_subnet.myapp_subnet_1: Destroying... [id=subnet-024e873f19b987ee0]
aws_default_security_group.myapp_sg: Destroying... [id=sg-016a544d965355413]
aws_default_security_group.myapp_sg: Destruction complete after 0s
aws_key_pair.ssh key: Destruction complete after 0s
aws_subnet.myapp_subnet_1: Destruction complete after 1s
aws_vpc.myapp_vpc: Destroying... [id=vpc-0284549ca1fb501a9]
aws_vpc.myapp_vpc: Destruction complete after 0s

Destroy complete! Resources: 7 destroyed.
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```

2 Verify Terraform state

After destroying resources, check your Terraform state files:

cat terraform.tfstate

cat terraform.tfstate.backup

```
@23-22411-007-oss →/workspaces/Lab-11 (main) $ cat terraform.tfstate
cat terraform.tfstate.backup
    "ipv6_cidr_block_network_border_group": "",
    "ipv6_ipam_pool_id": "",
    "ipv6_netmask_length": 0,
    "main_route_table_id": "rtb-03a3eca8af45bd348",
    "owner_id": "218496845154",
    "region": "me-central-1",
    "tags": {
      "Name": "dev-vpc"
    },
    "tags_all": {
      "Name": "dev-vpc"
    }
  },
  "sensitive_attributes": [],
  "identity_schema_version": 0,
  "identity": {
    "account_id": "218496845154",
    "id": "vpc-0284549ca1fb501a9",
    "region": "me-central-1"
  },
  "private": "eyJzY2h1bWFFdmVyc2lvbiI6IjEifQ=="
}
]
}
],
"check_results": null
}
```

3 Check for sensitive files in Git

Ensure you haven't accidentally committed .pem keys, .tfstate files, or AWS credentials:

git status

```
}  
@23-22411-007-oss →/workspaces/Lab-11 (main) $ git status  
On branch main  
Your branch is up to date with 'origin/main'.  
  
Untracked files:  
  (use "git add <file>..." to include in what will be committed)  
    .gitignore  
    .terraform.lock.hcl  
    entry-script.sh  
    id_ed25519.pub  
    locals.tf  
    main.tf  
    main.tfs  
    terraform.tfvars  
  
nothing added to commit but untracked files present (use "git add" to track)  
@23-22411-007-oss →/workspaces/Lab-11 (main) $
```